

Maritime CargoService: Sprint 1

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 09, 2026

1. Introduction

Il punto di partenza di questo sprint è l'architettura logica (recuperabile al seguente [link](#)^o) e la formalizzazione dei requisiti (recuperabile al seguente [link](#)^o).

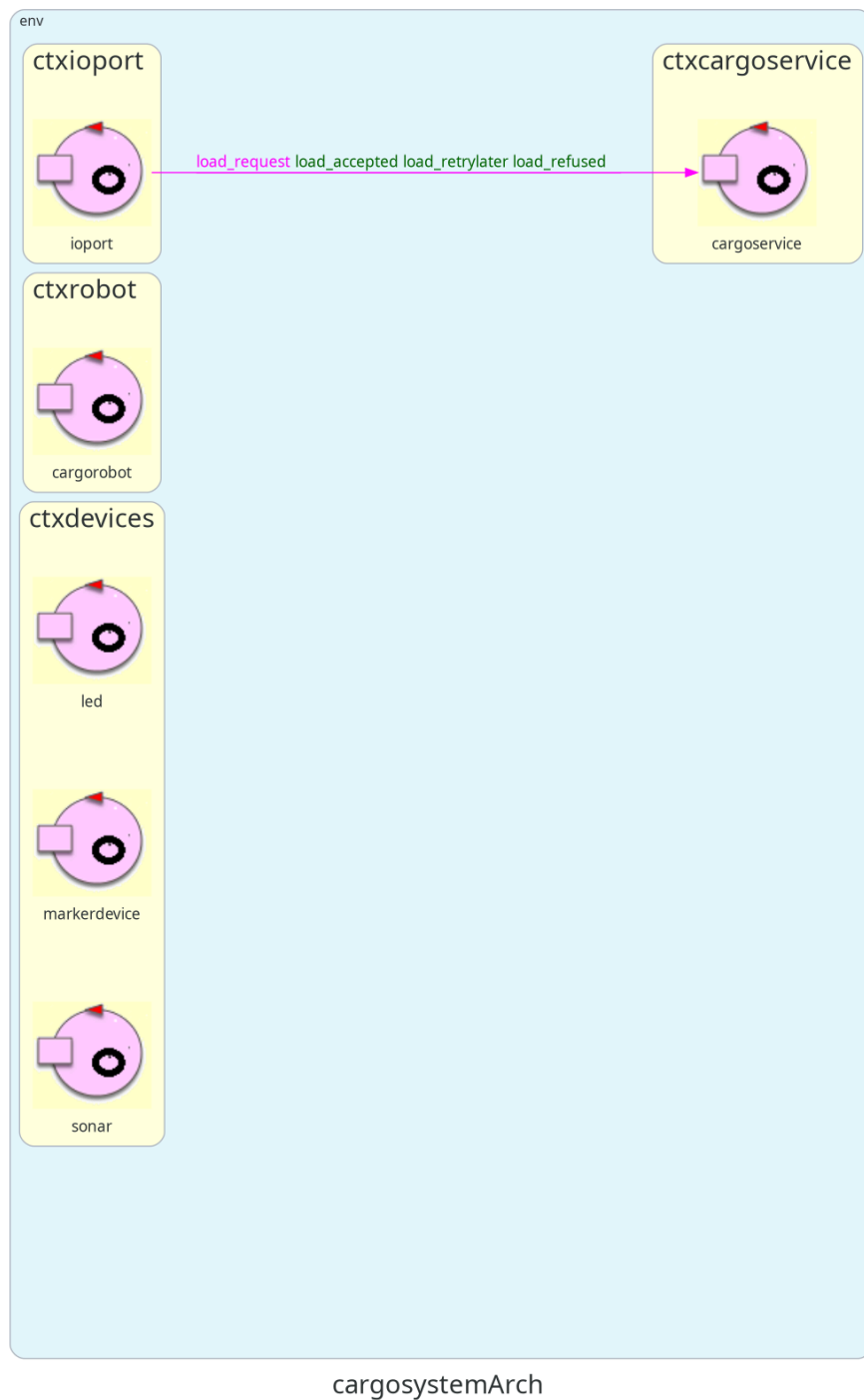


Figure 1: Architettura definita nello Sprint0.

Si riporta di seguito il goal dello sprint 1:

DA AGGIORNARE

Realizzare un primo prototipo eseguibile del **cargoservice** che ne implementi il comportamento descritto dai requisiti mediante collaboratori simulati. Poiché l'obiettivo principale è validare la logica del servizio, il movimento del robot viene astratto come collaborazione simulata, assumendo che l'operazione di deposito termini con esito positivo. Questa scelta consente di verificare prima il coordinamento tra i componenti e rimandare a una fase successiva il controllo fisico del robot e la gestione dei percorsi.

Al termine dello Sprint1 il sistema dovrà essere in grado di:

- ricevere una **load_request** proveniente dall'IOPort;
- verificare lo stato della hold, dell'IOPort e del servizio;
- produrre una delle tre risposte previste:
 - **load_accepted(slotID);**
 - **load_retrylater;**
 - **load_refused;**
- mantenere il modello logico della hold e la prenotazione dello slot;
- gestire gli stati **engaged** e **disengaged**;
- pilotare il LED in funzione dello stato del sistema;
- superare i test funzionali definiti nello Sprint0.

In questa fase il cargorobot, il sonar, il markerdevice e l'IOPort saranno rappresentati tramite collaboratori simulati.

Proposta di aggiunta: =Evoluzione dal modello dello sprint 0

Prendendo come punto di partenza il modello sviluppato alla fine dello sprint0, dobbiamo poi ecc.. ecc..

2. Requirements

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Gli slot1-4 rappresentano le aree della stiva riservate per immagazzinare ciascuno un container.
- Lo slot5 rappresenta un'area in cui il cargorobot deve temporaneamente depositare un container, prima di posizionarlo in uno degli slot1-4. Durante la sosta temporanea, un dispositivo 'marker' etichetta il container con un codice a barre identificativo e segnala quando l'attività di marcatura è completata.
- L'IOPort è un dispositivo dotato di un pulsante e di un display. Il pulsante viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display viene utilizzato per mostrare la risposta alla richiesta e lo stato attuale della stiva.

- Il sensore associato all'IOPort è un dispositivo (un sonar) utilizzato per rilevare la presenza di un container, quando misura una distanza D , tale che $D < D_{\text{FREE}}/2$, per un tempo ragionevole (ad esempio 3 secondi).
- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente
 - il messaggio "**Out of service**" se il sensore sonar misura la distanza (del container dal sonar stesso) $D > D_{\text{FREE}}$ per almeno 3 secondi (possibile guasto del sonar)

3. Requirement analysis

L'analisi dettagliata del dominio e dei requisiti è già stata affrontata nello Sprint 0, in questa fase ci concentriamo esclusivamente sul ciclo di gestione delle richieste di carico (**core business**) del sistema.

Lo svolgimento corretto di tali azioni presuppone tuttavia dell'esistenza di altri componenti del sistema fisici o in forma simulata (**sonar**, **LED**, **cargorobot**, **markerdevice**, **IOPort** come web GUI) non ancora sviluppati. La loro realizzazione concreta è pianificata per gli sprint successivi. I componenti "mock" che replicheranno, per test, il comportamento di quelli mancanti verranno evoluti rispetto alla formalizzazione QAK già avvenuta in fase di Sprint0 (recuperabile al seguente [link](#)^o).

4. Problem analysis

Come emerso dall'analisi dei requisiti, questo componente funge da **orchestratore**: coordina le operazioni degli altri componenti del sistema al fine di eseguire le procedure di carico. Inoltre, le entità sono distribuite su quattro nodi separati ma, per non rallentare la prototipazione, tutti i componenti verranno rappresentati nello stesso nodo

(rappresentato da un Context), tenendo in considerazione che gli attori non condividono memoria, e comunicano tra loro tramite scambio di messaggi.

4.1. Rappresentazione dello stato interno della stiva

In questa fase la gestione della stiva viene rappresentata tramite un componente di dominio dedicato, modellato come **POJO** Java, invece che come attore autonomo.

La scelta è motivata dal fatto che la stiva non rappresenta un'entità attiva del sistema, ma una struttura dati che descrive lo stato interno dell'ambiente: posizione degli slot, ostacoli, IOPort, home del robot e stato di occupazione degli slot.

Separare questa logica dal `cargoservice` permette di rispettare il principio di singola responsabilità: il `cargoservice` coordina il servizio, mentre la classe `Hold` gestisce esclusivamente le informazioni relative alla stiva.

Inoltre, lo stato iniziale della stiva viene caricato da un file di configurazione JSON, così da evitare valori hard-coded nel codice e rendere più semplice modificare la disposizione dell'ambiente senza cambiare la logica applicativa.

Il file di configurazione è disponibile al seguente link: [configurazione hold](#)^o

Il codice della classe `Hold` è disponibile al seguente link: [codice Hold](#)^o

4.2. Analisi delle Interazioni

Di seguito si riporta una formalizzazione delle altre interazioni modellate interpretando e applicando opportune scelte progettuali sui requisiti del sistema.

- Sappiamo che il `cargoservice` deve interrogare la `hold` per sapere se c'è un posto libero ed eventualmente riservarlo. Il `cargoservice` non può quindi procedere finché la `hold` non ha risposto in modo affermativo (in caso di slot libero) o negativo (in caso di stiva piena). Scegliamo quindi una comunicazione di tipo Request/Reply.
- Sarà anche necessario fornire un meccanismo per liberare lo slot una volta che il timeout di 30 secondi sarà scaduto. In questo caso, non è necessario attendere una risposta, quindi si utilizza una Dispatch.

```
// CargoService <-> Hold
Request get_slot      : getSlot(none)
Reply  slot_reserved  : slotReserved(SLOTID) for get_slot
Reply  hold_full      : holdFull(none) for get_slot
Dispatch free_slot    : freeSlot(SLOTID) // NON NECESSARIO, guarda risposta
committente sprint0
```

- Da requisiti, sappiamo che il sonar deve misurare continuamente la distanza del container dal sonar stesso. Nasce quindi la necessità di dover trasmettere queste misurazioni al `cargoservice`. Per isolare la responsabilità del sonar a semplice “misuratore e trasmettitore” di informazione, viene naturale formalizzare tale comunicazione come un Event, ovvero un messaggio broadcast che verrà ascoltato da chi interessato (in questo caso **cargoservice**)

```
Event    sonardata      : distance(D)
```

- L'invio di comandi al led è delegato al cargoservice. Non è necessario attendere una risposta, quindi utilizziamo una Dispatch.

```
Dispatch led_ctrl      : ledCmd(CMD)
// CMD: "on", "off", "blink"
```

- In questa fase, il movimento del robot sarà simulato, ma il cargoservice deve comunque attendere la fine dell'operazione per poter procedere con la

successiva fase del ciclo operativo (ossia richiedere al **markerdevice** l'etichettatura in **slot5**, confermare lo stoccaggio e tornare allo stato **disengaged**). Usiamo quindi la Request/Reply per assicurarci che il cargoservice attenda la fine del task.

```
// CargoService <-> Robot
Request robot_move : robotMove(TARGET)
Reply  robot_done  : robotDone(none) for robot_move
```

- In modo analogo, il cargoservice deve interrogare il markerdevice per sapere quando l'operazione di etichettatura è terminata, non potendo procedere finché il markerdevice non ha risposto. Usiamo quindi la Request/Reply.

```
// CargoService <-> MarkerDevice
Request mark_container : markContainer(none)
Reply  marking_done   : markingDone(none) for mark_container
```

4.3. Rappresentazione dell'attore cargoservice come Macchina a Stati Finiti

L'attore cargoservice è, già da Sprint 0, inteso come una Macchina a Stati Finiti che utilizza variabili interne per:

- memorizzare se la porta di ingresso risulta occupata (IOPortOccupied)
- se il servizio è attualmente operativo (ServiceWorking)
- lo stato logico del servizio (CargoState)
- l'identificativo dello slot eventualmente riservato (ReservedSlotId).

Per quanto riguarda le transizioni della FSM, si estende il comportamento già realizzato in Sprint0.

```
Context ctxprototype ip [host="localhost" port=8050] // CONTESTO UNICO

QActor cargoservice context ctxprototype {
    [# // VARIABILI INTERNE
        var IOPortOccupied = false
        var ServiceWorking = true
        var CargoState     = "disengaged"
```

```
var ReservedSlotId = -1
#]
```

- Quando viene ricevuta una richiesta **load_request**, il cargoservice prima di prenotare uno slot verifica che:
 1. il servizio non sia già impegnato nella gestione di un altro container
 2. il sistema risulti funzionante
 3. che la porta di ingresso sia libera.

Se tutte le condizioni risultino soddisfatte viene interrogata la Hold per richiedere la prenotazione di uno slot libero, il cargoservice memorizza l'identificativo dello slot riservato, passa allo stato **engaged** e informa il client dell'avvenuta accettazione della richiesta.

Viene inoltre attivato il LED in modalità **blink** e viene comunicato l'ID dello slot (da mostrare in futuro sul display). Nel caso in cui il magazzino risulti completamente occupato cargoservice comunicherà il rifiuto della richiesta, rimanendo disponibile per eventuali.

Il timer di 30 secondi viene avviato quando il sistema entra nello stato **engaged**. Se il container non viene posizionato nell'area del sensore entro tale intervallo, il sistema passa allo stato **disengaged**, libera lo slot precedentemente riservato e spegne il LED.

```
State s0 initial {
    println("cargoservice | STARTED") color magenta
}
Goto disengaged

State disengaged {
    println("cargoservice | DISENGAGED: waiting for requests...") color
blue
}
Transition t0
    whenRequest load_request -> handle_load_request
    whenEvent    sonardata    -> handle_sonar

State engaged {
    println("cargoservice | ENGAGED: waiting for container deposit...")
color blue
}
Transition t0
    whenTime    30000          -> handle_deposit_timeout
    whenRequest load_request -> handle_load_request
    whenEvent    sonardata    -> handle_sonar

//GESTIONE RICHIESTE

State handle_load_request {
    printCurrentMessage color yellow
```



```

        // Verifica precondizioni per accettare la richiesta
        if [# CargoState == "engaged" || !ServiceWorking || IOPortOccupied
#] {
            replyTo load_request with load_retrylater : loadRetryLater(none)
        } else {
            // Interroga la Hold per uno slot libero
            request hold -m get_slot : getSlot(none)
        }
    }
    Transition t0
        whenReply slot_reserved -> accept_request
        whenReply hold_full      -> refuse_request

    State accept_request {
        onMsg(slot_reserved : slotReserved(ID)) {
            [#
                ReservedSlotId = payloadArg(0).toInt()
                CargoState      = "engaged"
            #]
            forward ledmock -m led_ctrl : ledCmd(blink)
            [# val SlotName = "slot$ReservedSlotId" #]
            replyTo load_request with load_accepted : loadAccepted($SlotName)
        }
    }
    Goto engaged

    State refuse_request {
        replyTo load_request with load_refused : loadRefused(none)
    }
    Goto disengaged

```

- Il cargoservic interpreta gli eventi emessi dal sonar in tre modi principali:
1. Viene rilevata la presenza di un container entro una certa distanza dal sensore (per almeno 3s) e viene impostata a true la variabile interna **IOPortOccupied** per indicare che l'IOPort è occupata.
 2. Viene rilevata una distanza superiore a DFREE (per almeno 3s). Il sistema viene in questo caso messo nello stato **Out of service**. Viene quindi aggiornata la variabile interna **ServiceWorking**.
 3. La distanza misurata non implica nessun cambiamento di stato. In questo caso il sistema continua a funzionare normalmente.

Al termine dell'elaborazione dell'evento il cargoservic valuta se siano contemporaneamente soddisfatte tutte le condizioni necessarie per iniziare la movimentazione del container. Il robot viene attivato solamente quando il servizio si trova nello stato engaged, il container è stato effettivamente depositato e il sistema risulta operativo.

```
//GESTIONE SONAR
```

```

State handle_sonar {
    onMsg(sonardata : distance(D)) {
        [# val Dist = payloadArg(0).toInt() #]

        // D < 50 indica presenza del container (IOPort occupata)
        if [# Dist < 50 #] {
            [# IOPortOccupied = true #]
        } else {
            [# IOPortOccupied = false #]
        }

        // D > DFREE (es. > 150) per un intervallo prolungato indica
condizione OUT OF SERVICE
        if [# Dist > 150 #] {
            [# ServiceWorking = false #]
            println("cargoservice | Sonar D > DFREE ($Dist) -> System OUT
OF SERVICE!") color red
        } else {
            if [# !ServiceWorking #] {
                println("cargoservice | Sonar D <= DFREE ($Dist) ->
System WORKING again!") color green
            }
            [# ServiceWorking = true #]
        }
    }
}

// Se il container viene posato durante lo stato engaged e il sistema è
operativo, il robot può iniziare
Goto do_robot_job if [# IOPortOccupied && CargoState == "engaged" &&
ServiceWorking #] else returnToState

State returnToState {}
Goto engaged if [# CargoState == "engaged" #] else disengaged

```

-Analisi

```

// GESTIONE ROBOT E MARKER

State do_robot_job {
    println("cargoservice | Container deposited! Moving to slot5...")
color magenta
    request cargorobotmock -m robot_move : robotMove(slot5)
}
Transition t0
    whenReply robot_done -> mark_container

State mark_container {
    println("cargoservice | At slot5. Asking markerdevice to mark...")
color magenta
    request markerdevice -m mark_container : markContainer(none)
}
Transition t0
    whenReply marking_done -> move_to_reserved_slot

```

```

    State move_to_reserved_slot {
        println("cargoservice | Marked! Moving to slot$ReservedSlotId...")
        color magenta
        [# val TargetSlot = "slot$ReservedSlotId" #]
        request cargorobotmock -m robot_move : robotMove($TargetSlot)
    }
    Transition t0
        whenReply robot_done -> finish_job

    State finish_job {
        println("cargoservice | Job completed!") color green
        [# CargoState = "disengaged" #]
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged

    State handle_deposit_timeout {
        println("cargoservice | Deposit timeout! Freeing slot.") color red

        [# CargoState = "disengaged" #]
        forward hold -m free_slot : freeSlot($ReservedSlotId)
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged
}

```

4.4. cargorobot

Come concordato con il committente si è scelto di utilizzare il simulatore di ambiente virtuale VirtualRobot26. Come descritto nella documentazione di smartrobot26, per poter pianificare la ricerca del percorso occorre che sia definita una mappa di contesto. Per costruirla si utilizza una stringa che poi andrà a denotare tale matrice. Conformemente all'immagine fornitaci dal committente, la mappa di contesto avrà la seguente forma:

[link](#)^o

5. Test plans

6. Deployment

Per procedere al deployment del prototipo è necessario eseguire i seguenti passaggi:

1. Clonare il repository del progetto da GitHub:
2. Eseguire all'interno della directory del progetto il comando `./gradlew build`
3. Avviare il prototipo eseguendo il comando `./gradlew run`

7. Pagina di sintesi

7.1. Architettura finale del prototipo

L'architettura finale del prototipo sviluppato durante questo Sprint è riportata nella figura seguente.

Per maggiori dettagli sul modello implementato si rimanda a [ProblemAnalysis](#), mentre i test sviluppati sono descritti in [Test Plan](#).

Questa architettura rappresenta il risultato finale dello Sprint 1 e costituirà il punto di partenza per lo Sprint 2.

7.2. Verifica della pianificazione

La pianificazione prevista per lo Sprint 1 è stata rispettata. Non si sono verificati scostamenti significativi rispetto agli obiettivi inizialmente prefissati.

7.3. Goal dello Sprint 2

L'obiettivo principale dello Sprint 2 sarà incrementare il livello di realismo del prototipo sostituendo i componenti simulati con le rispettive implementazioni reali.

In particolare, le attività previste sono:

- sostituire il **cargorobotmock** con l'attore reale interfacciato a **VirtualRobot26**;
- realizzare l'**IOPort** come Web GUI, consentendo l'interazione dell'utente tramite browser;
- integrare i nuovi componenti mantenendo invariata l'architettura ad attori del sistema;
- verificare il corretto funzionamento del sistema mediante un aggiornamento dei test funzionali.

8. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340